Slajd 1. Strona tytułowa

Dzień dobry, nazywam się Piotr Słupski. Chciałbym przedstawić moją pracę inżynierską pod tytułem „Zaawansowana aplikacja webowa do organizacji zadań”.

Praca została przygotowana pod kierunkiem promotora, dr. Leszka Grocholskiego.

Slajd 2. Spis treści

Na początku przedstawię cel i zakres pracy, porównanie podobnych aplikacji oraz wymagania funkcjonalne i niefunkcjonalne. Następnie omówię ogólną architekturę aplikacji, technologie frontendu i backendu, autentykację i autoryzację oraz wybór bazy danych. W dalszej części zaprezentuję schemat bazy danych, diagram komponentów i diagramy przepływu danych. Na końcu przedstawię testy, przejdę do demonstracji aplikacji i podsumowania, a prezentację zamknie bibliografia.

Slajd 3. Cel pracy

Celem głównym niniejszej pracy było zaprojektowanie oraz zaimplementowanie aplikacji internetowej wspierającej użytkowników indywidualnych w organizacji zarówno złożonych zadań, jak i codziennych obowiązków.

Cele szczegółowe obejmowały przede wszystkim zaprojektowanie interfejsu graficznego, opra-

cowanie logiki frontendowej i backendowej obsługującej zarządzanie zadaniami w formach widocznych na ekranie, implementację obsługi kont użytkowników oraz zaprojektowanie bazy danych.

Slajd 4. Zakres pracy

Zakres pracy obejmował:

- analizę podobnych aplikacji oraz wymagań użytkownika;
- omówienie technologii wykorzystanych w projekcie;
- zaprojektowanie interfejsu użytkownika na podstawie scenariuszy przypadków użycia;
- implementację kodu po stronie frontendu i backendu;
- przygotowanie testów jednostkowych wybranych funkcjonalności aplikacji.

Slajd 5. Podobne aplikacje

Przed rozpoczęciem projektowania przeanalizowałem trzy aplikacje do organizacji zadań i zarządzania projektami: Trello, ClickUp oraz Asanę.

Trello opiera organizację pracy na tablicach Kanban i kartach zadań, które można uzupełniać o listy kontrolne. ClickUp łączy wiele widoków pracy ze wspólnymi dokumentami, pomiarem czasu i pulpitami analitycznymi. Asana wspiera planowanie projektów za pomocą harmonogramów, zależności między zadaniami oraz kontroli obciążenia zespołu.

Z porównania wyciągnąłem przede wszystkim wniosek dotyczący zakresu własnego projektu: użytkownik indywidualny potrzebuje szybkiego dostępu do bieżących zadań i związanych z nimi informacji. Dlatego w mojej aplikacji połączyłem taski, checklisty, notatki i projekty oraz możliwość przypinania wybranych elementów do dashboardu. Nacisk położyłem na czytelność interfejsu i powiązania między tymi elementami, pozostawiając rozbudowane raportowanie i mechanizmy zespołowe poza zakresem pracy.

Źródło do powtórzenia: Oficjalne strony aplikacji Trello, ClickUp i Asana, wymienione w bibliografii

Slajd 6. Wymagania funkcjonalne

Wymagania funkcjonalne określiłem na podstawie analizy potrzeb użytkownika indywidualnego, porównania podobnych aplikacji oraz przygotowanych historii użytkownika. Opisywały one czynności, które użytkownik powinien móc wykonać w systemie, i obejmowały:

- logowanie i tworzenie konta za pośrednictwem zewnętrznego dostawcy;

- obsługę operacji CRUD dla tasków, notatek, checklist i projektów;
- przypinanie wybranych elementów do dashboardu;
- tworzenie powiązań pomiędzy różnymi elementami aplikacji;
- wyświetlanie tasków projektu w formie listy lub tablicy Kanban;
- wyszukiwanie, filtrowanie i sortowanie elementów.

Slajd 7. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne dotyczyły jakości działania aplikacji oraz sposobu jej dalszego utrzymania. Obejmowały:

- czytelny, responsywny i spójny graficznie interfejs;
- dostęp użytkownika wyłącznie do własnych danych;
- sprawną walidację danych przesyłanych do API;
- strukturę kodu umożliwiającą dalszy rozwój aplikacji.

Slajd 8. Ogólna architektura aplikacji

Aplikacja została zaprojektowana w architekturze klient–serwer. Klientem jest interfejs uruchamiany w przeglądarce, który prezentuje dane, obsługuje działania użytkownika i wysyła żądania.

Część serwerowa została zrealizowana w Next.js za pomocą Route Handlerów. Odbiera ona żądania, sprawdza sesję użytkownika, waliduje dane, wykonuje logikę aplikacji i komunikuje się z bazą MongoDB.

Komunikacja przebiega w modelu żądanie–odpowiedź. Przykładowo formularz wysyła do odpowiedniego endpointu dane nowego taska, serwer sprawdza ich poprawność i zapisuje dokument, a następnie zwraca rezultat operacji. Frontend nie łączy się bezpośrednio z bazą danych, dzięki czemu reguły dostępu oraz operacje zapisu pozostają po stronie serwera.

Zaletą tego rozwiązania jest rozdzielenie odpowiedzialności. Zmiany interfejsu nie muszą wpływać na model danych, a wspólna warstwa API zapewnia jedno miejsce dla walidacji, kontroli właściciela dokumentu i obsługi błędów. Taki podział zwiększa bezpieczeństwo, ułatwia testowanie poszczególnych części oraz pozwala rozwijać aplikację bez przebudowy całego systemu.

Źródło do powtórzenia: Canva, Creating a Data Flow Diagram: How-tos, Templates, and Tips

---

Slajd 9. Technologie frontendu

Lucide to biblioteka spójnych ikon wykorzystanych w przyciskach, nawigacji i oznaczeniach poszczególnych elementów interfejsu.

Tailwind CSS to narzędzie do stylowania za pomocą klas narzędziowych, które ułatwiło przygotowanie spójnego i responsywnego wyglądu aplikacji.

Zod to biblioteka służąca do definiowania schematów oraz walidowania po stronie API danych przesyłanych z formularzy.

Vitest to środowisko uruchamiania testów, wykorzystane do sprawdzania wybranych formularzy, komponentów i mechanizmów aplikacji.

TypeScript jest statycznie typowanym rozszerzeniem języka JavaScript, które pomaga wykrywać niezgodności typów już podczas tworzenia aplikacji.

Slajd 10. Technologie backendu

Node.js jest środowiskiem uruchomieniowym JavaScript, w którym wykonywana jest serwerowa część aplikacji.

NextAuth.js, obecnie rozwijany jako Auth.js, odpowiada za logowanie przez zewnętrznych dostawców oraz tworzenie i obsługę sesji użytkownika.

Mongoose jest biblioteką pośredniczącą w komunikacji z MongoDB, umożliwiającą definiowanie modeli i schematów dokumentów aplikacji.

Slajd 11. Autentykacja i autoryzacja

Obsługę logowania zaimplementowałem przy użyciu Auth.js oraz dostawców OAuth Google i GitHub. Identyfikatory klientów i sekrety są pobierane ze zmiennych środowiskowych, a dostawca jest włączany tylko wtedy, gdy jego konfiguracja jest kompletna. Dzięki temu dane uwierzytelniające nie znajdują się bezpośrednio w kodzie aplikacji.

Adapter MongoDB zapisuje informacje potrzebne do obsługi użytkowników i kont dostawców, natomiast sesja wykorzystuje strategię JWT. Konta Google i GitHub są traktowane osobno nawet wtedy, gdy dostawcy zwrócą ten sam adres e-mail, co zapobiega automatycznemu łączeniu kont bez potwierdzenia użytkownika.

Callback sesji pobiera identyfikator użytkownika z tokenu i umieszcza go w polu `session.user.id`. Route Handlery odczytują następnie ten identyfikator przez wspólną funkcję `getCurrentUserId`. Brak poprawnej sesji powoduje zwrócenie odpowiedzi HTTP 401, a operacje na danych są wykonywane z filtrem `ownerId`. W rezultacie zalogowany użytkownik może pobierać

i modyfikować wyłącznie należące do niego taski, checklisty, notatki oraz projekty.

Źródło do powtórzenia: S.-T. Sun, K. Beznosov, The Devil is in the (Implementation) Details: An Empirical Analysis of OAuth SSO Systems

---

Slajd 12. Wybór bazy danych

Podczas wyboru bazy danych porównałem relacyjny model MySQL z dokumentowym modelem MongoDB. MySQL przechowuje informacje w tabelach o zdefiniowanych kolumnach, a relacje pomiędzy rekordami realizuje za pomocą kluczy głównych i obcych. Takie podejście zapewnia ścisłą kontrolę integralności danych i dobrze sprawdza się w systemach o stabilnej strukturze oraz precyzyjnie określonych zależnościach.

MongoDB przechowuje dane jako dokumenty o strukturze zbliżonej do obiektów JSON. Pozwala zarówno osadzać powiązane dane wewnątrz dokumentu, jak i tworzyć relacje przez identyfikatory innych dokumentów. W aplikacji wykorzystałem oba rozwiązania: elementy checklisty są zagnieżdżone w jej dokumencie, natomiast relacje pomiędzy taskami, projektami, notatkami i przypięciami są zapisywane przy użyciu identyfikatorów.

MongoDB zostało wybrane ze względu na strukturę danych aplikacji. Projekty przechowują własne kolumny Kanban, checklisty zawierają tablice elementów, a poszczególne typy dokumentów mają różne pola opcjonalne i powiązania. Model dokumentowy pozwala odwzorować te obiekty bez tworzenia wielu tabel pomocniczych i ułatwia rozwijanie ich struktury. Elastyczność bazy została uzupełniona modelami Mongoose, które definiują oczekiwane pola, typy i reguły zapisu, dlatego dane nadal pozostają kontrolowane po stronie aplikacji.

Slajd 13. Schemat bazy danych

Schemat przedstawia najważniejsze kolekcje znajdujące się w bazie danych aplikacji oraz zapisane w nich pola. Widoczne powiązania pokazują, w jaki sposób dokumenty użytkowników, tasków, projektów, checklist, notatek i przypięć odwołują się do siebie za pomocą identyfikatorów.

Slajd 14. Diagram komponentów aplikacji

Diagram przedstawia najważniejsze komponenty systemu: interfejs użytkownika, warstwę API, logikę serwerową, bazę danych oraz zewnętrznych dostawców logowania. Strzałki pokazują kierunki komunikacji pomiędzy tymi elementami podczas wykonywania operacji przez użytkownika.

Slajd 15. Diagram przepływu danych poziomu 0

Diagram poziomu 0 przedstawia całą aplikację jako jeden proces komunikujący się z użytkownikiem, dostawcą autentykacji i bazą danych. Pokazuje granice systemu oraz główne kierunki przepływu informacji bez prezentowania jego wewnętrznych procesów.

Źródło do powtórzenia: Canva, Creating a Data Flow Diagram: How-tos, Templates, and Tips

---

Slajd 16. Diagram przepływu danych poziomu 1

Diagram poziomu 1 rozwija poprzedni schemat i rozdziela aplikację na procesy odpowiedzialne za logowanie oraz obsługę tasków, projektów, notatek i checklist. Pokazuje również, które procesy odczytują dane z odpowiednich kolekcji lub zapisują w nich zmiany.

Źródło do powtórzenia: Canva, Creating a Data Flow Diagram: How-tos, Templates, and Tips

---

Slajd 17. Testy

Na slajdzie przedstawiono test schematu walidacji danych wykorzystywanych podczas tworzenia taska. Funkcja `taskCreateSchema.safeParse` sprawdza kompletny zestaw pól i zwraca wynik bez przerywania działania testu wyjątkiem, natomiast asercje `expect` weryfikują powodzenie walidacji oraz poprawne usunięcie zbędnych spacji z tytułu, statusu, tagów i tytułu nowej checklisty.

Strukturę zestawu testów tworzą funkcje `describe` oraz `it`: pierwsza grupuje przypadki dotyczące schematów taska, a druga opisuje pojedynczy sprawdzany scenariusz. Tak przygotowany test potwierdza nie tylko przyjęcie prawidłowego payloadu, ale także jego normalizację przed dalszym wykorzystaniem przez API.

Slajd 18. Demonstracja aplikacji

DEMONSTRACJA

Slajd 19. Podsumowanie

W ramach pracy zaprojektowałem i zaimplementowałem działającą aplikację internetową wspierającą indywidualną organizację tasków, checklist, notatek oraz projektów. Zrealizowany system łączy responsywny interfejs z bezpieczną warstwą serwerową, dokumentową bazą danych oraz mechanizmami logowania, walidacji i testowania.

Slajd 20. Bibliografia

Ahmad M. O., Dennehy D., Conboy K., Oivo M., *Kanban in Software Engineering: A Systematic Mapping Study*, "Journal of Systems and Software" 2018, Vol. 137, s. 96–113, `https://doi.org/10.1016/j.jss.2017.11.045` (dostęp: 31.07.2026 r.).

Albiorix Technology, *Web Application Architecture Layers*, grafika przedstawiająca warstwy architektury aplikacji internetowej, [b.r.], `https://www.albiorixtech.com/wp-content/uploads/2025/12/web-application-architecture-layers.jpg` (dostęp: 03.02.2026 r.).

Asana Inc., *Asana*, strona internetowa aplikacji do zarządzania pracą, [b.r.], `https://asana.com` (dostęp: 17.07.2026 r.).

Atlassian, *Trello*, strona internetowa aplikacji Kanban do zarządzania zadaniami, [b.r.], `https://trello.com` (dostęp: 17.07.2026 r.).

Auth.js, *Auth.js*, dokumentacja biblioteki do obsługi uwierzytelniania, [b.r.], https://authjs.dev (dostęp: 17.07.2026 r.).

Auth.js, *Auth.js Core Reference*, dokumentacja callbacków biblioteki Auth.js, [b.r.], https://authjs.dev/reference/core#callbacks (dostęp: 16.08.2026 r.).

Canva, *Creating a Data Flow Diagram: How-tos, Templates, and Tips*, artykuł dotyczący tworzenia diagramów przepływu danych, [b.r.], https://www.canva.com/online-whiteboard/data-flow-diagrams/ (dostęp: 03.02.2026 r.).

ClickUp, *ClickUp*, strona internetowa aplikacji do zarządzania projektami, [b.r.], https://clickup.com (dostęp: 17.07.2026 r.).

Dean J., *Web Programming with HTML5, CSS, and JavaScript*, Jones & Bartlett Learning, Burlington 2017.

Frontend vs Backend, grafika przedstawiająca porównanie warstwy frontendowej i backendowej, [b.r.], https://d1zruf9db62p8s.cloudfront.net/2025/08/Frontend-vs-Backend.webp (dostęp: 02.02.2026 r.).

Microsoft, *TypeScript Documentation*, dokumentacja języka TypeScript, [b.r.], https://www.typescriptlang.org/docs/ (dostęp: 25.07.2026 r.).

MongoDB Inc., *Welcome to the MongoDB Docs*, dokumentacja systemu MongoDB, [b.r.], https://www.mongodb.com/docs/ (dostęp: 02.02.2026 r.).

Mongoose, *Schemas*, dokumentacja biblioteki Mongoose dotycząca schematów danych, [b.r.], https://mongoosejs.com/docs/guide.html (dostęp: 16.08.2026 r.).

Oracle, *MySQL Documentation*, dokumentacja systemu zarządzania bazą danych MySQL, [b.r.], https://dev.mysql.com/doc/ (dostęp: 25.07.2026 r.).

React, *Built-in React Hooks*, dokumentacja hooków biblioteki React, [b.r.], https://react.dev/reference/react/hooks (dostęp: 16.08.2026 r.).

React, *React: The Library for Web and Native Interfaces*, dokumentacja biblioteki React, [b.r.], https://react.dev (dostęp: 28.01.2026 r.).

Sun S.-T., Beznosov K., *The Devil is in the (Implementation) Details: An Empirical Analysis of OAuth SSO Systems*, [w:] *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, 2012, s. 378–390, DOI: 10.1145/2382196.2382238, https://doi.org/10.1145/2382196.2382238 (dostęp: 31.07.2026 r.).

Tailwind Labs, *Tailwind CSS*, dokumentacja frameworka CSS, [b.r.], https://tailwindcss.com (dostęp: 28.01.2026 r.).

Testing Library, *React Testing Library API*, dokumentacja API biblioteki React Testing Library, [b.r.], https://testing-library.com/docs/react-testing-library/api/ (dostęp: 03.08.2026 r.).

Vercel, *Next.js Docs*, dokumentacja frameworka Next.js, [b.r.], https://nextjs.org/docs (dostęp: 29.01.2026 r.).

Vercel, *Route Handlers*, dokumentacja Next.js dotycząca obsługi endpointów, [b.r.], https://nextjs.org/docs/app/getting-started/route-handlers (dostęp: 16.08.2026 r.).

Vitest, *Writing Tests*, dokumentacja pisania testów w Vitest, [b.r.], https://vitest.dev/guide/learn/writing-tests.html (dostęp: 03.08.2026 r.).

Zod, *Defining schemas*, dokumentacja biblioteki Zod dotycząca definiowania schematów walidacji, [b.r.], https://zod.dev/api?id=property (dostęp: 16.08.2026 r.).